
MOBILE DEEP LEARNING INFERENCE FRAMEWORK FOR ANDROID-BASED SHORT-CIRCUIT TRACE CLASSIFICATION

Supervisor: Prof. Junho Bang

Presenter: Hadi Nazari

IT Applied System Engineering



전북대학교
JEONBUK NATIONAL UNIVERSITY

INTRODUCTION

01

Electrical fires
often caused
by short
circuits

02

- Need to distinguish PSCT vs SSCT

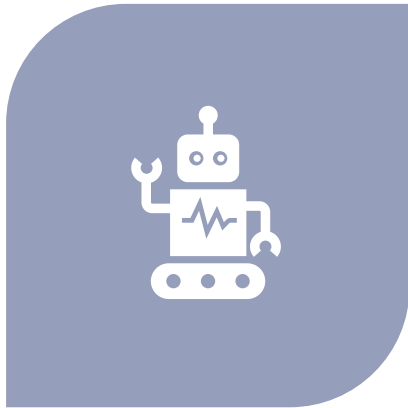
03

- Traditional methods are manual and subjective

04

- AI enables automated classification

OBJECTIVE



- DEVELOP ANDROID-BASED CLASSIFICATION SYSTEM

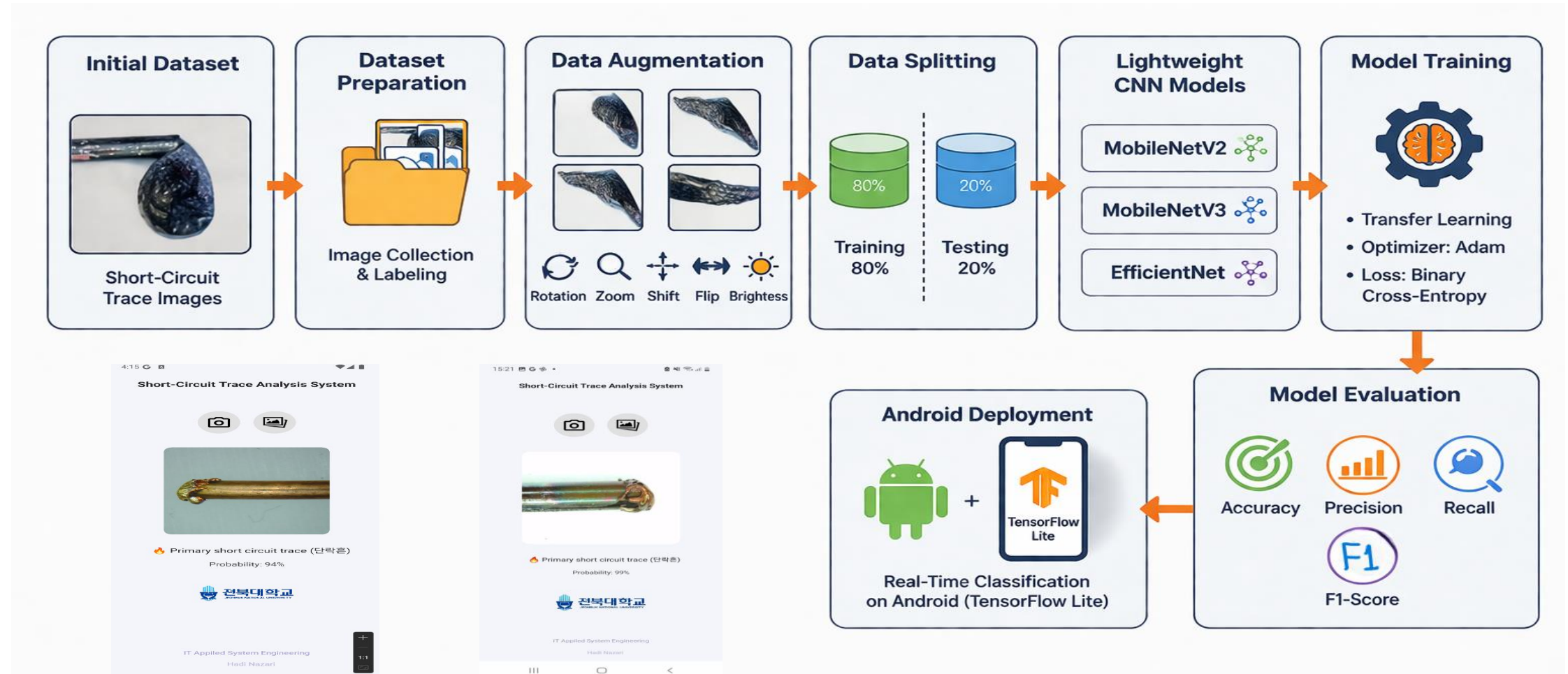


- USE LIGHTWEIGHT CNN MODELS

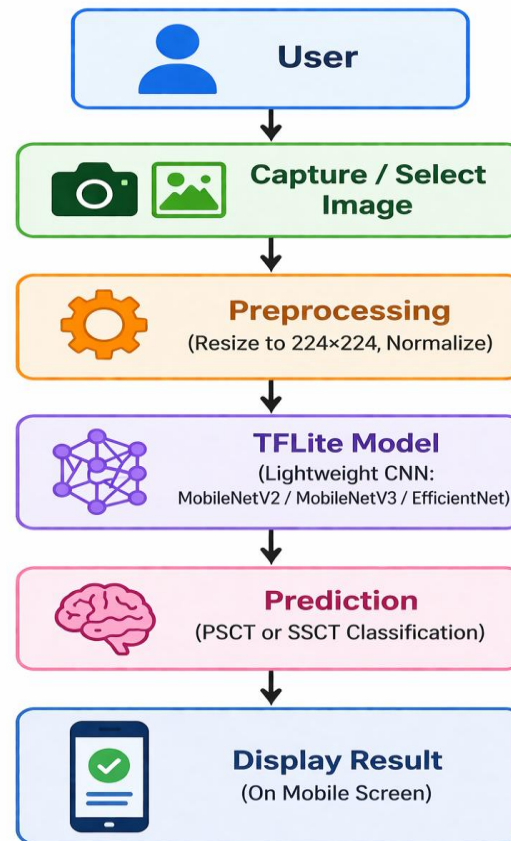


- ENABLE REAL-TIME MOBILE INFERENCE

PROPOSED FRAMEWORK FOR ELECTRIC FIRE SHORT-CIRCUIT TRACE CLASSIFICATION



ANDROID SYSTEM OVERVIEW



DATASET



Total images: 752



- Classes: primary short-circuit traces (PSCT), secondary short-circuit traces (SSCT)



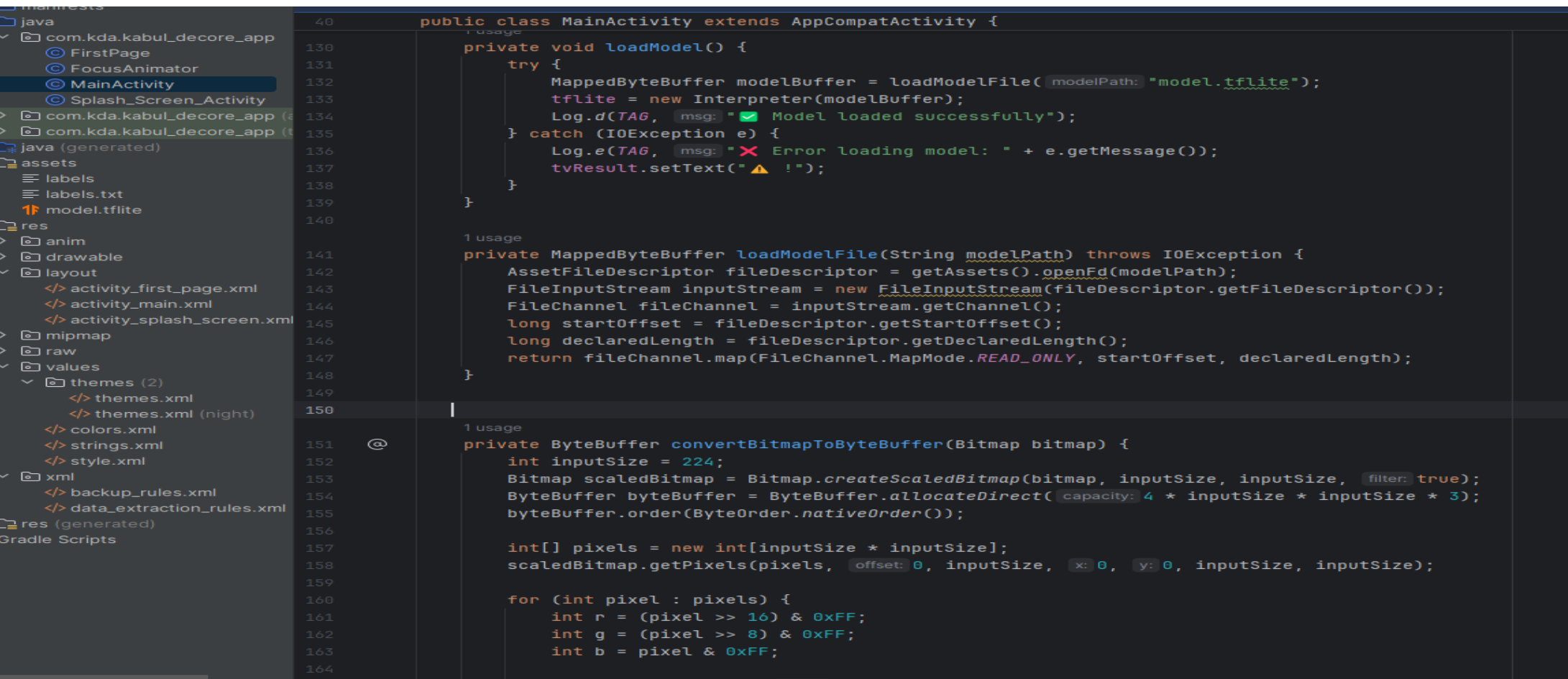
- Split: 80% training, 20% testing



- Augmentation: rotation, zoom, flip, brightness

PREPROCESSING

- Input image resized to: 224×224 pixels
- Converted into ByteBuffer format
- Pixel normalization applied for model compatibility



```
140 public class MainActivity extends AppCompatActivity {
141     private void loadModel() {
142         try {
143             MappedByteBuffer modelBuffer = loadModelFile("model.tflite");
144             TFLite = new Interpreter(modelBuffer);
145             Log.d(TAG, "Model loaded successfully");
146         } catch (IOException e) {
147             Log.e(TAG, "Error loading model: " + e.getMessage());
148             tvResult.setText("⚠ !");
149         }
150     }
151
152     private MappedByteBuffer loadModelFile(String modelPath) throws IOException {
153         AssetFileDescriptor fileDescriptor = getAssets().openFd(modelPath);
154         FileInputStream inputStream = new FileInputStream(fileDescriptor.getFileDescriptor());
155         FileChannel fileChannel = inputStream.getChannel();
156         long startOffset = fileDescriptor.getStartOffset();
157         long declaredLength = fileDescriptor.getDeclaredLength();
158         return fileChannel.map(FileChannel.MapMode.READ_ONLY, startOffset, declaredLength);
159     }
160
161     private ByteBuffer convertBitmapToByteBuffer(Bitmap bitmap) {
162         int inputSize = 224;
163         Bitmap scaledBitmap = Bitmap.createScaledBitmap(bitmap, inputSize, inputSize, true);
164         ByteBuffer byteBuffer = ByteBuffer.allocateDirect(4 * inputSize * inputSize * 3);
165         byteBuffer.order(ByteOrder.nativeOrder());
166
167         int[] pixels = new int[inputSize * inputSize];
168         scaledBitmap.getPixels(pixels, 0, inputSize, 0, 0, inputSize, inputSize);
169
170         for (int pixel : pixels) {
171             int r = (pixel >> 16) & 0xFF;
172             int g = (pixel >> 8) & 0xFF;
173             int b = pixel & 0xFF;
```



TensorFlow

MODEL CONVERSION (TENSORFLOW LITE)

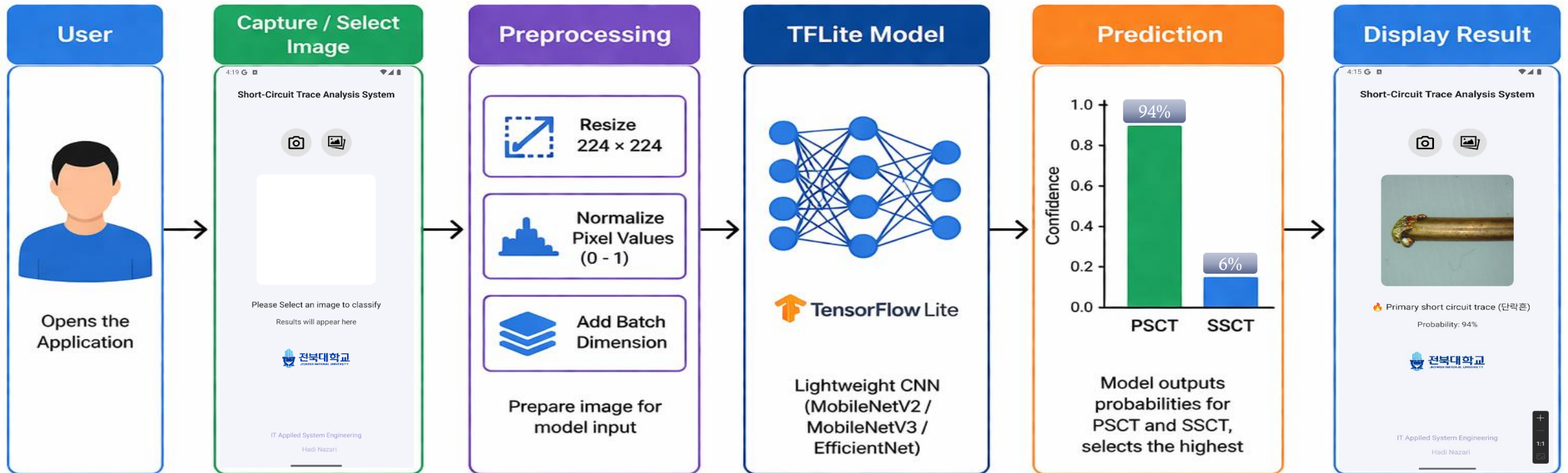
- After training, the best-performing model was converted to TensorFlow Lite (.tflite) format.
- This step enables:
 - Reduced model size
 - Faster inference
 - Compatibility with mobile devices

ANDROID IMPLEMENTATION

- The mobile application was developed using:
 - **Java**
 - **Android Studio**



SYSTEM OVERVIEW



RESULTS

EfficientNet: 96.05% accuracy

- MobileNetV3: stable performance
 - MobileNetV2: efficient and fast
-

APPLICATION INTERFACE

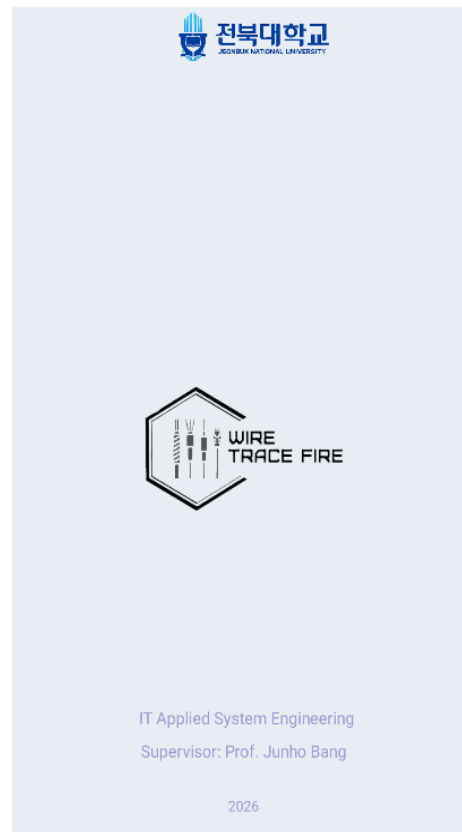
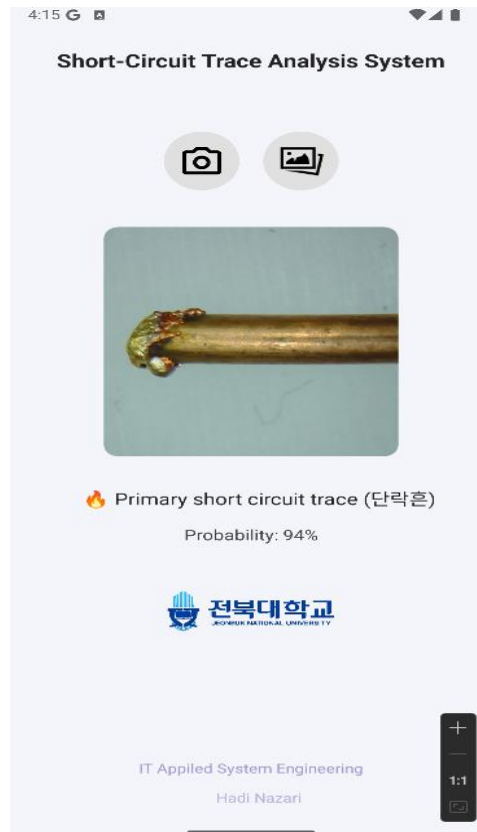


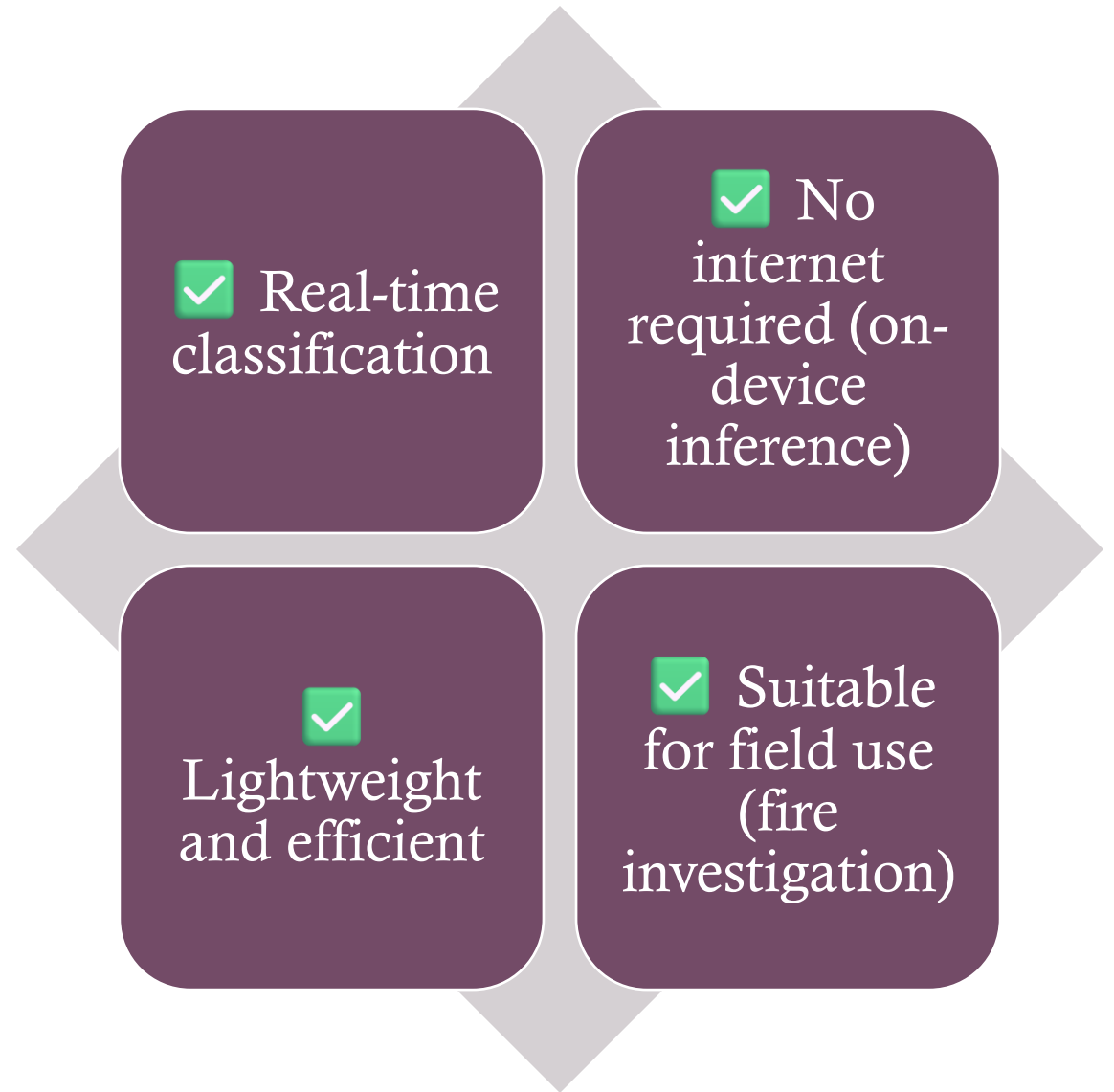
Image input (camera/gallery)

- Prediction result

- Probability output

- User-friendly UI

ADVANTAGES OF THE MOBILE IMPLEMENTATION



SUMMARY



Model trained in Python (Anaconda, TensorFlow/Keras)



Converted to TensorFlow Lite format



Integrated into Android Studio (Java)



Supports camera & gallery input



Performs real-time on-device classification



Displays predicted class and confidence